Interprocess Communications

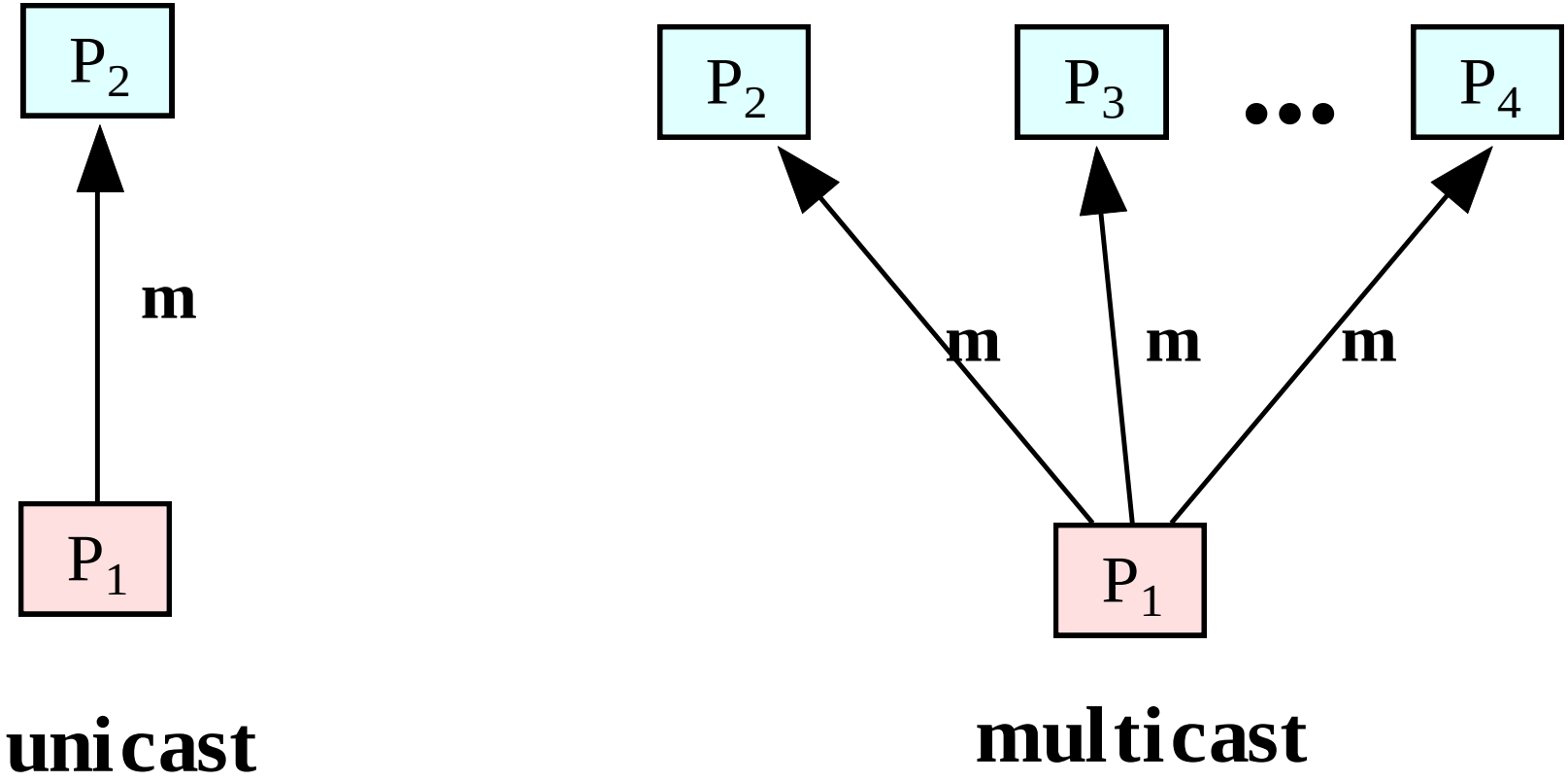
Interprocess Communications

- **Exchange of data** between two or more separate, independent **processes/threads**.
- Operating systems provide facilities/resources for inter-process communications (**IPC**), such as message queues, semaphores, and shared memory.
- Distributed computing systems make use of these facilities/resources to provide **application programming interface** (**API**) which allows IPC to be programmed at a higher level of abstraction. (e.g., send and receive)
- Distributed computing requires information to be exchanged among independent processes.

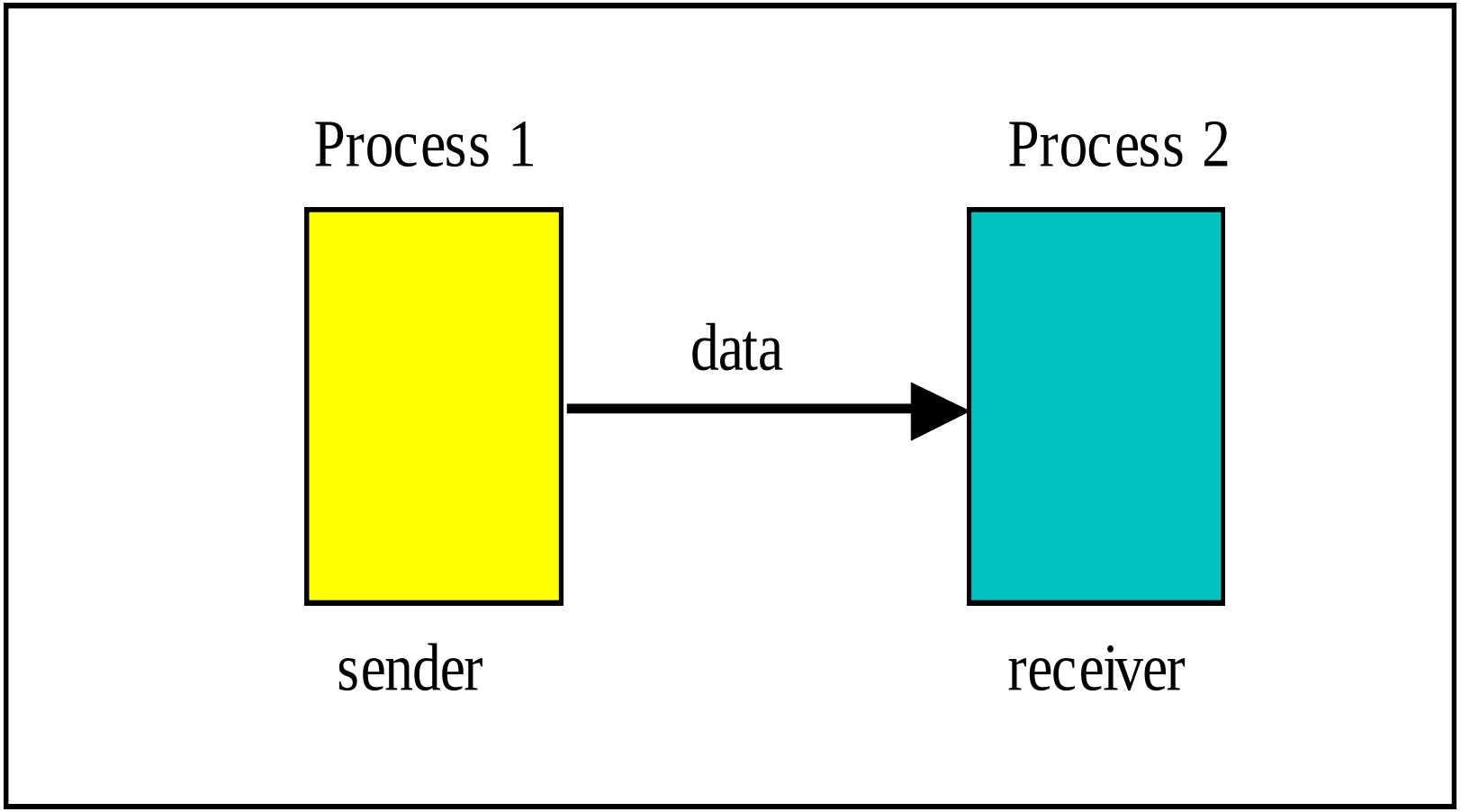
IPC – unicast and multicast

- In distributed computing, two or more processes engage in **IPC** using a **protocol** agreed upon by the **processes**. A process may be a sender at some points during a protocol, a receiver at other points.
- When communication is from one process to a single other process, the IPC is said to be a ***unicast***, e.g., **Socket communication**. When communication is from one process to a group of processes, the IPC is said to be a ***multicast***, e.g., **Publish/Subscribe Message model**, a topic that we will explore in a later chapter.

Unicast vs. Multicast



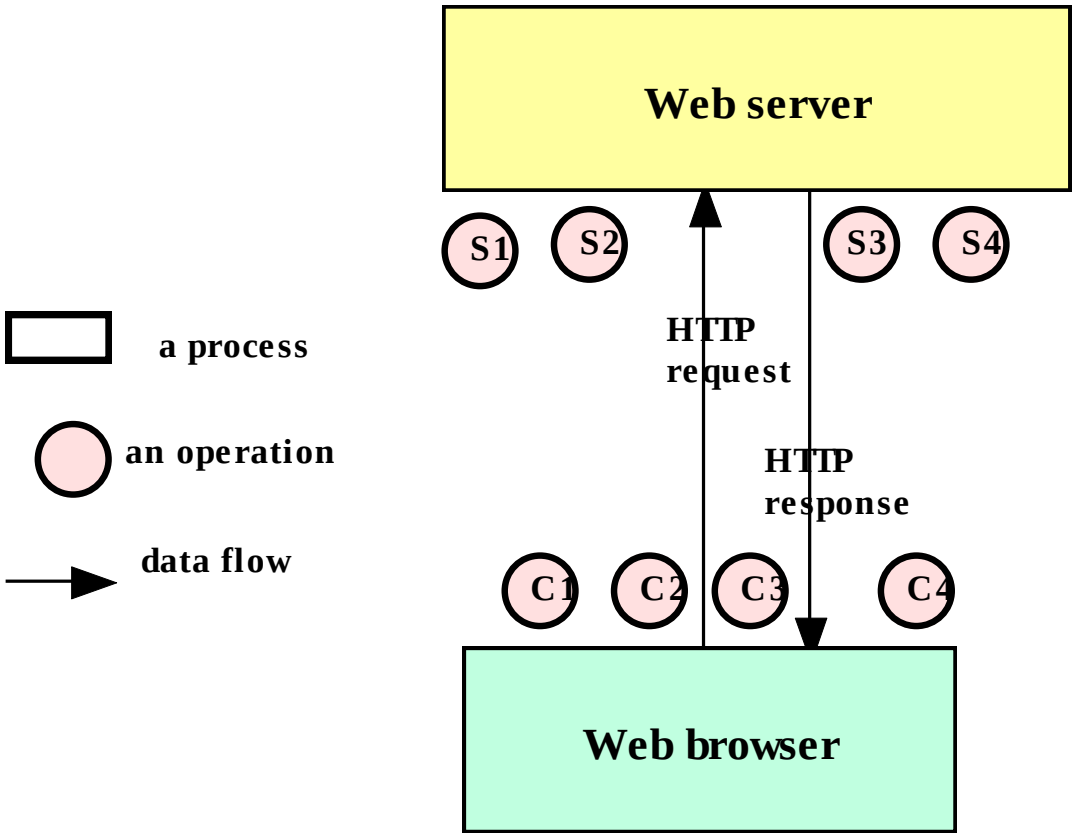
Interprocess Communications in Distributed Computing



Operations provided in an archetypal Interprocess Communications API

- **Receive** ([sender], message storage object)
- **Connect** (sender address, receiver address), for connection-oriented communication.
- **Send** ([receiver], message)
- **Disconnect** (connection identifier), for connection-oriented communication.

Interprocess Communication in basic HTTP



operations:
S1: accept connection
S2: receive (request)
S3: send (response)
s4 disconnect
C1: make connection
C2: send (request)
C3: receive (response)
C4: disconnect

Processing order: C1, S1, C2, S2, S3, C3, C4, S4

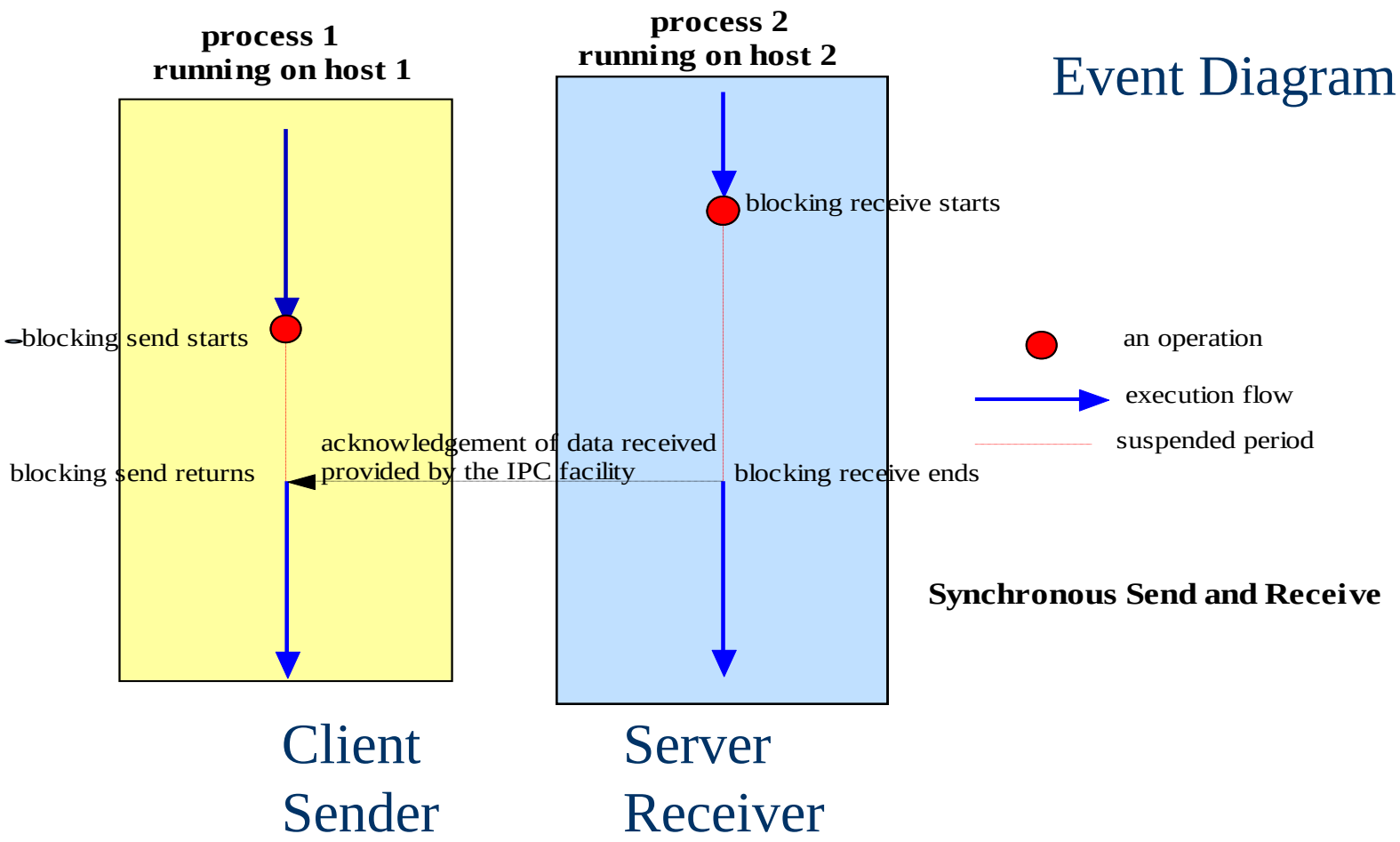
Event Synchronization

- Interprocess communication may require that the two processes synchronize their operations: one side sends, then the other receives until all data has been sent and received.
- Ideally, the send operation starts before the receive operation commences.
- In practice, the synchronization requires system support.

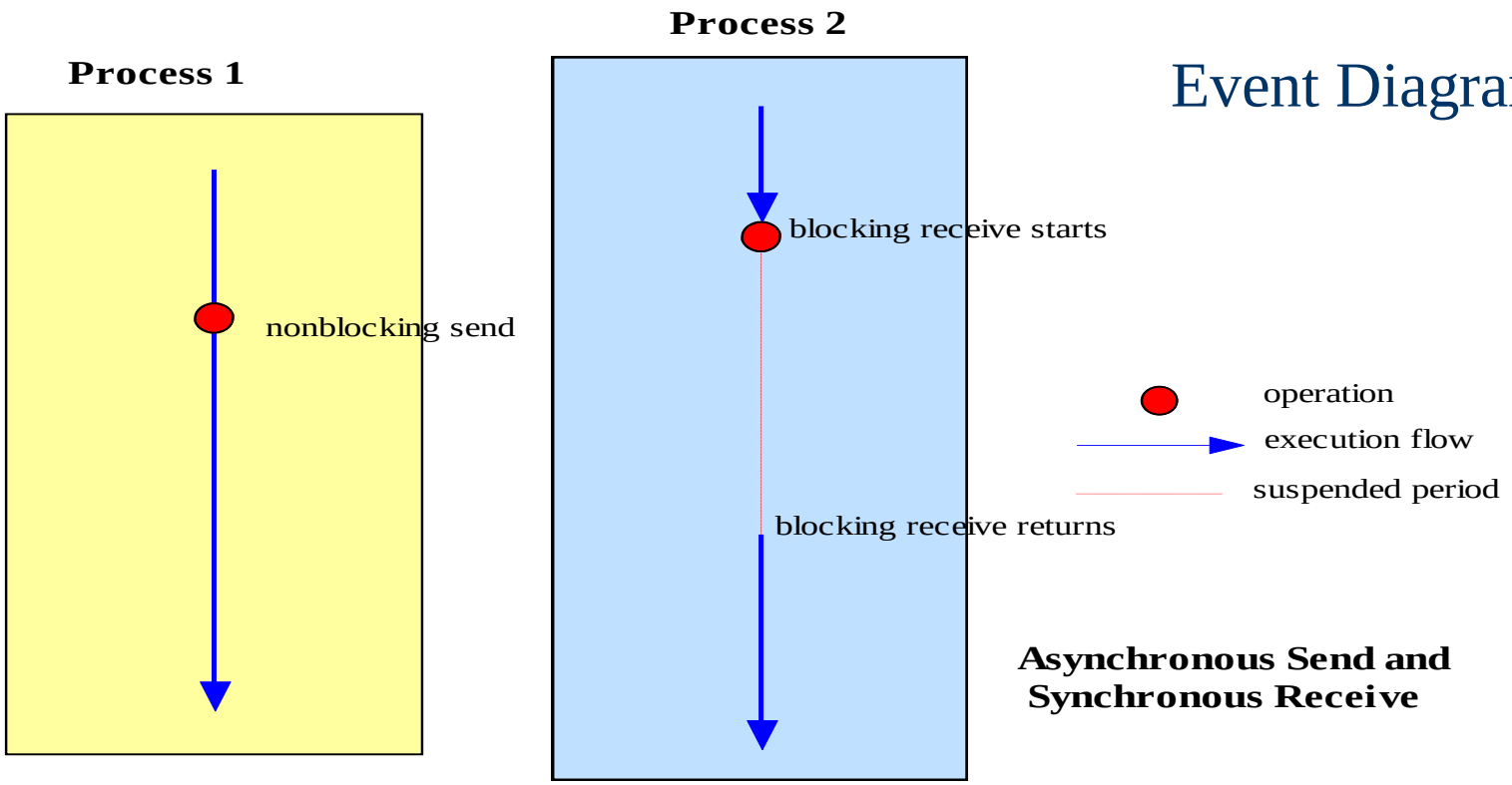
Synchronous vs. Asynchronous Communication

- **The IPC operations may provide the synchronization necessary using *blocking*. A blocking operation issued by a process will block further processing of the process until the operation is fulfilled.**
- **Alternatively, IPC operations may be asynchronous or nonblocking. An asynchronous operation issued by a process will not block further processing of the process. Instead, the process is free to proceed with its processing, and may optionally be notified by the system when the operation is fulfilled.**

Synchronous send and receive



Asynchronous send and synchronous receive

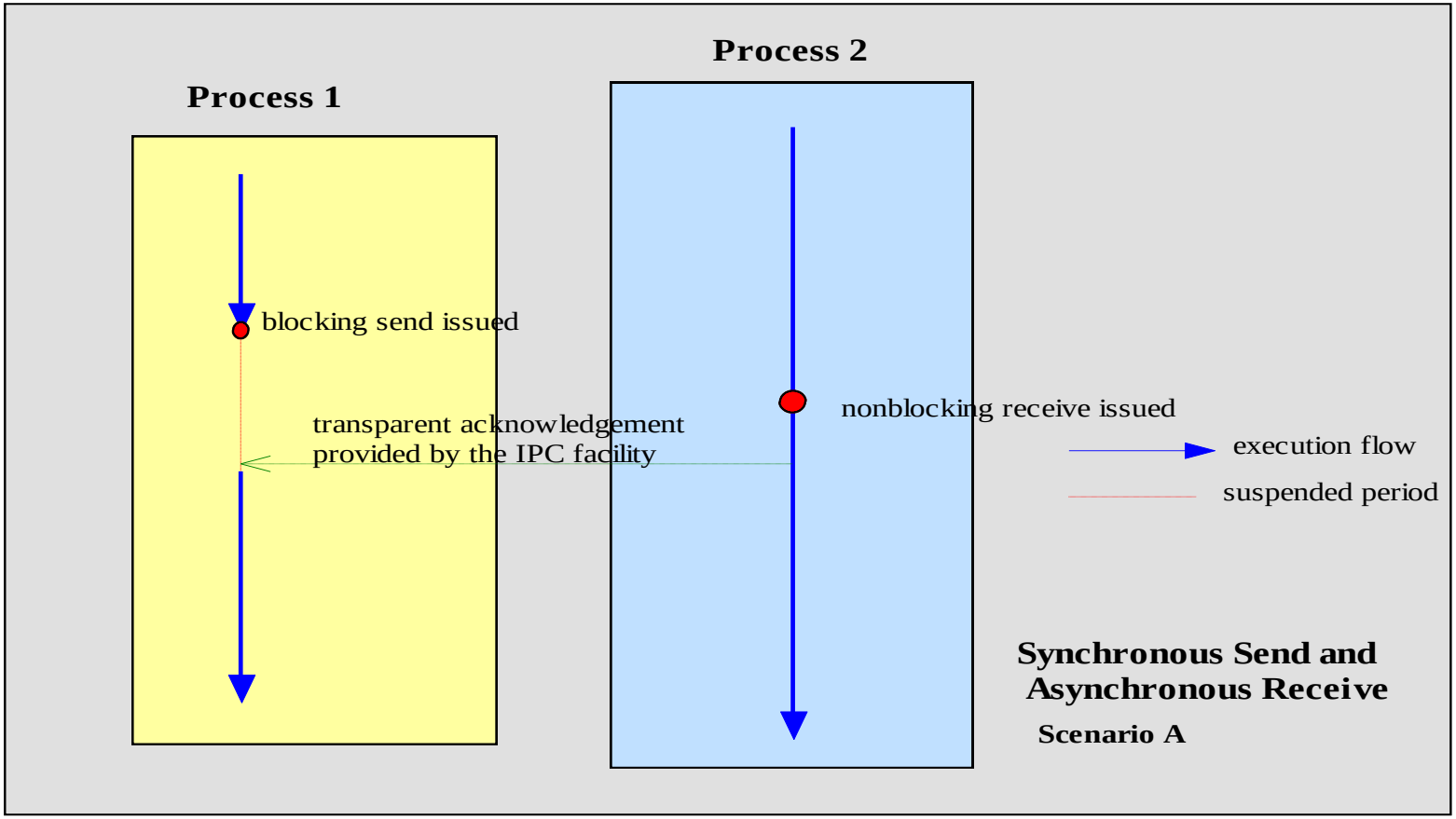


Event Diagram

Client
Sender

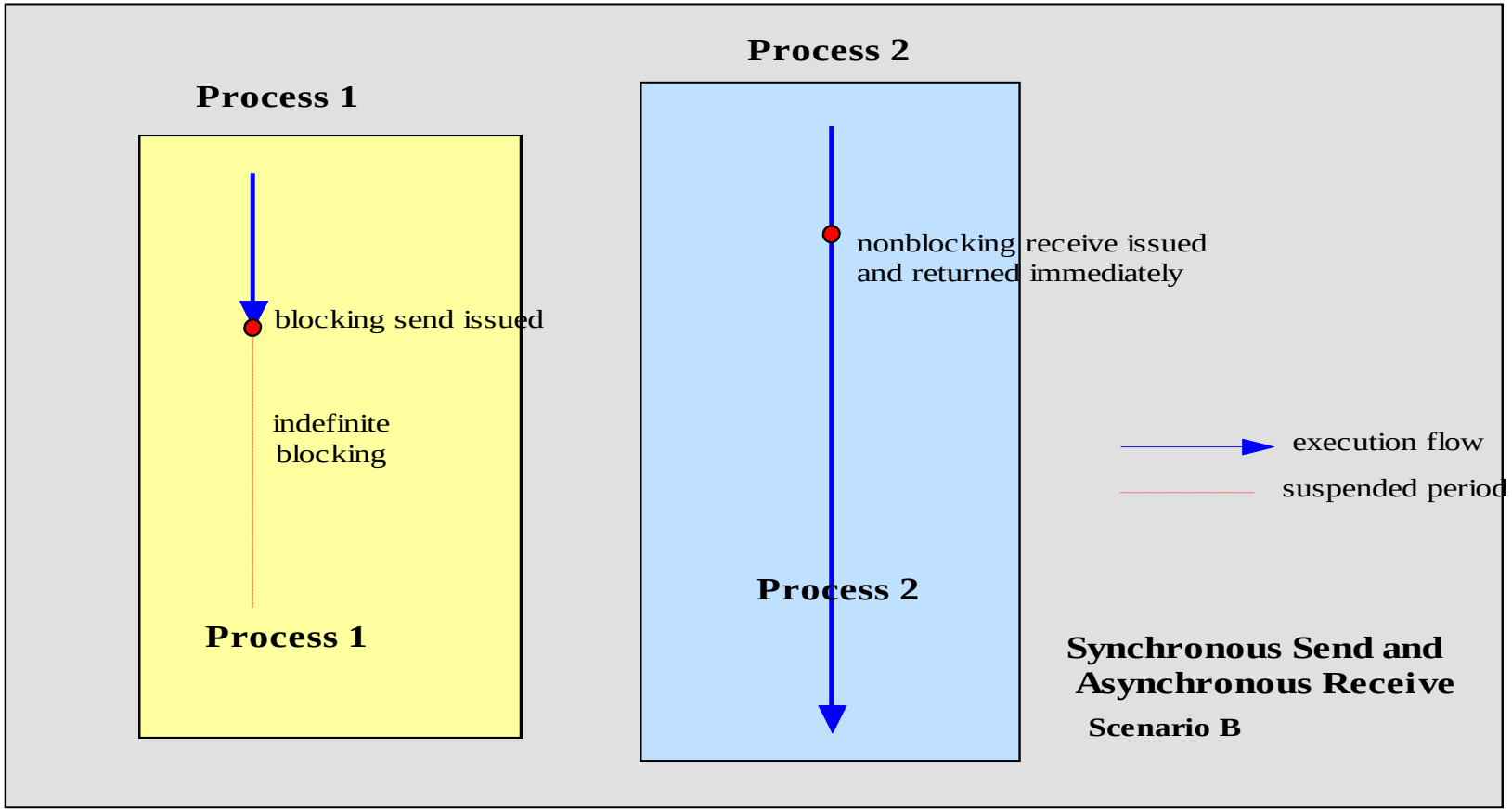
Server
Receiver

Synchronous send and Async. Receive - 1



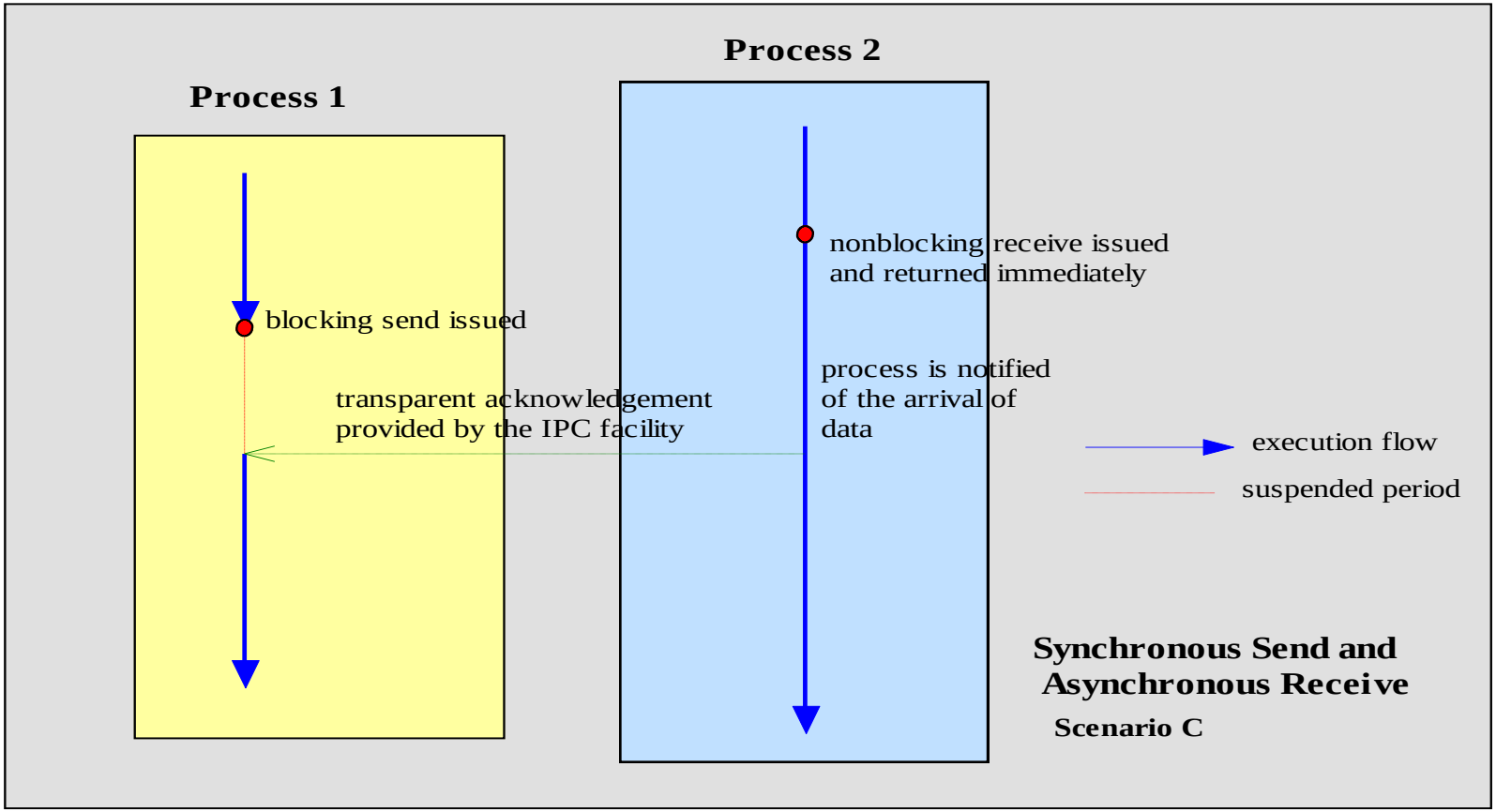
Data from P1 was received by P2
before issuing a non-blocking receive op in P2

Synchronous send and Async. Receive - 2



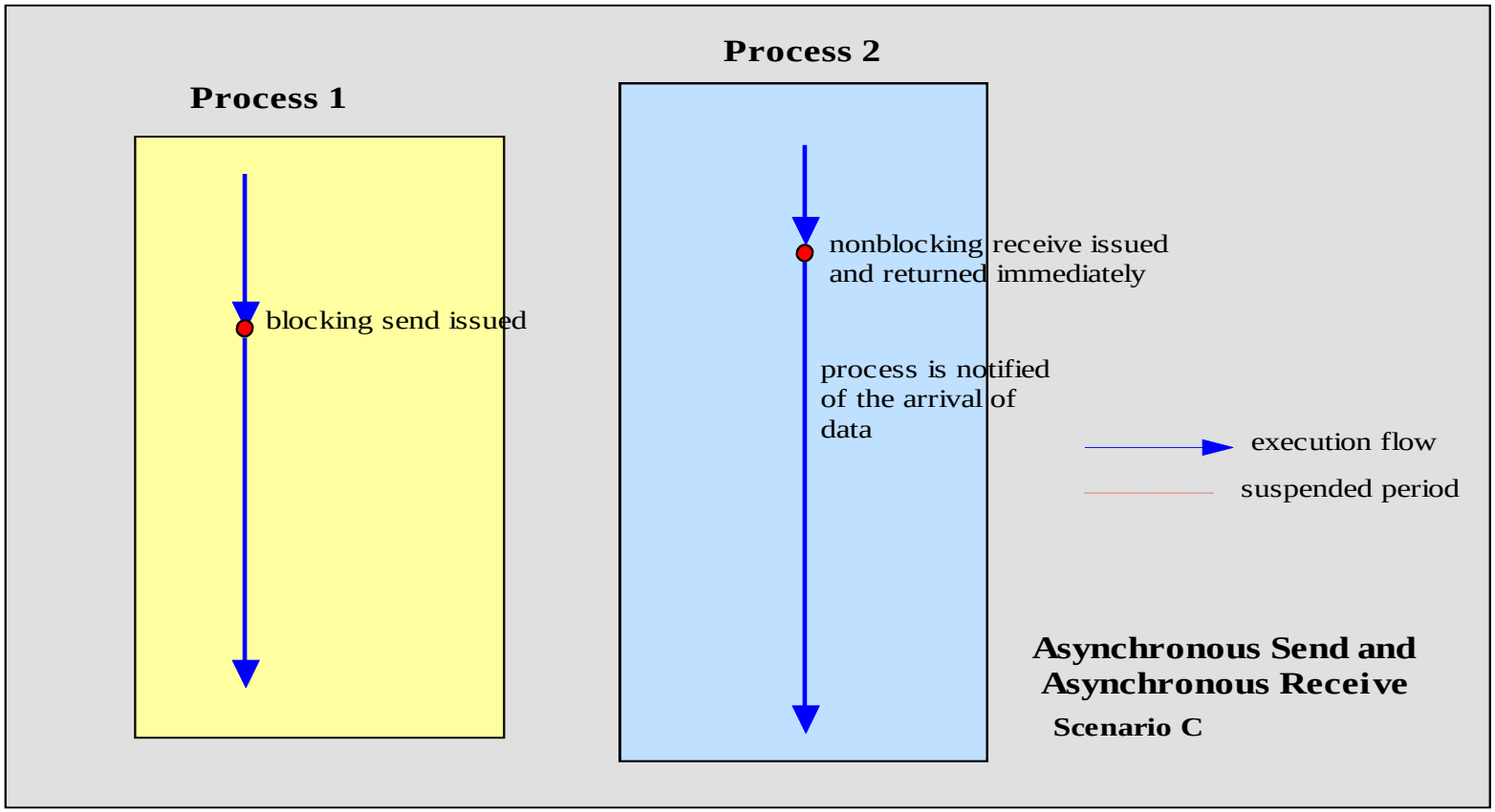
Data from P1 arrived to P2 **after** P2 issued a non-blocking receive op

Synchronous send and Async. Receive - 3



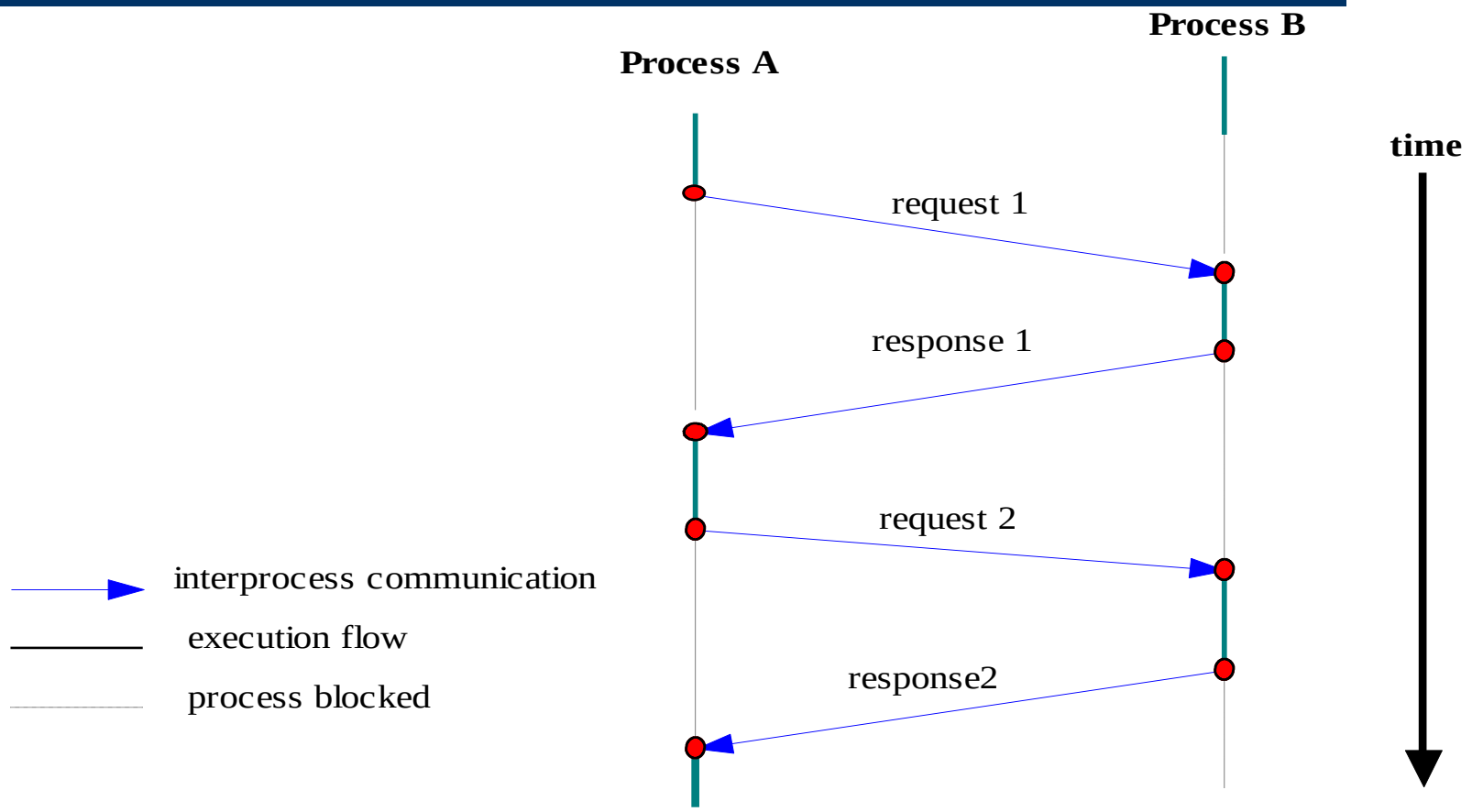
Data from P1 arrived to P2 **before** P2 issues a non-blocking receive op. P2 is notified of the arrival of data

Asynchronous send and Asynchronous receive



Does P1 need an acknowledgement from P2?

Event diagram

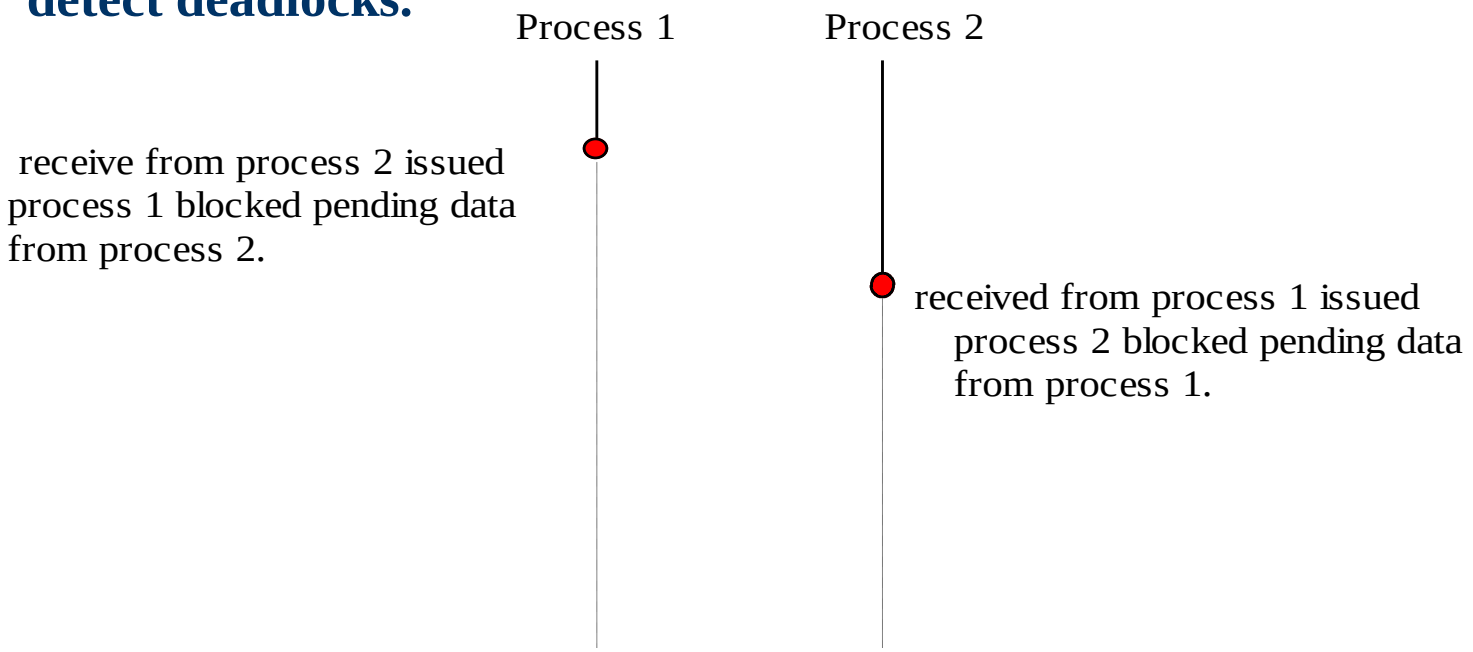


Event diagram for a protocol

Synchronous send and receive

Blocking, deadlock, and timeouts

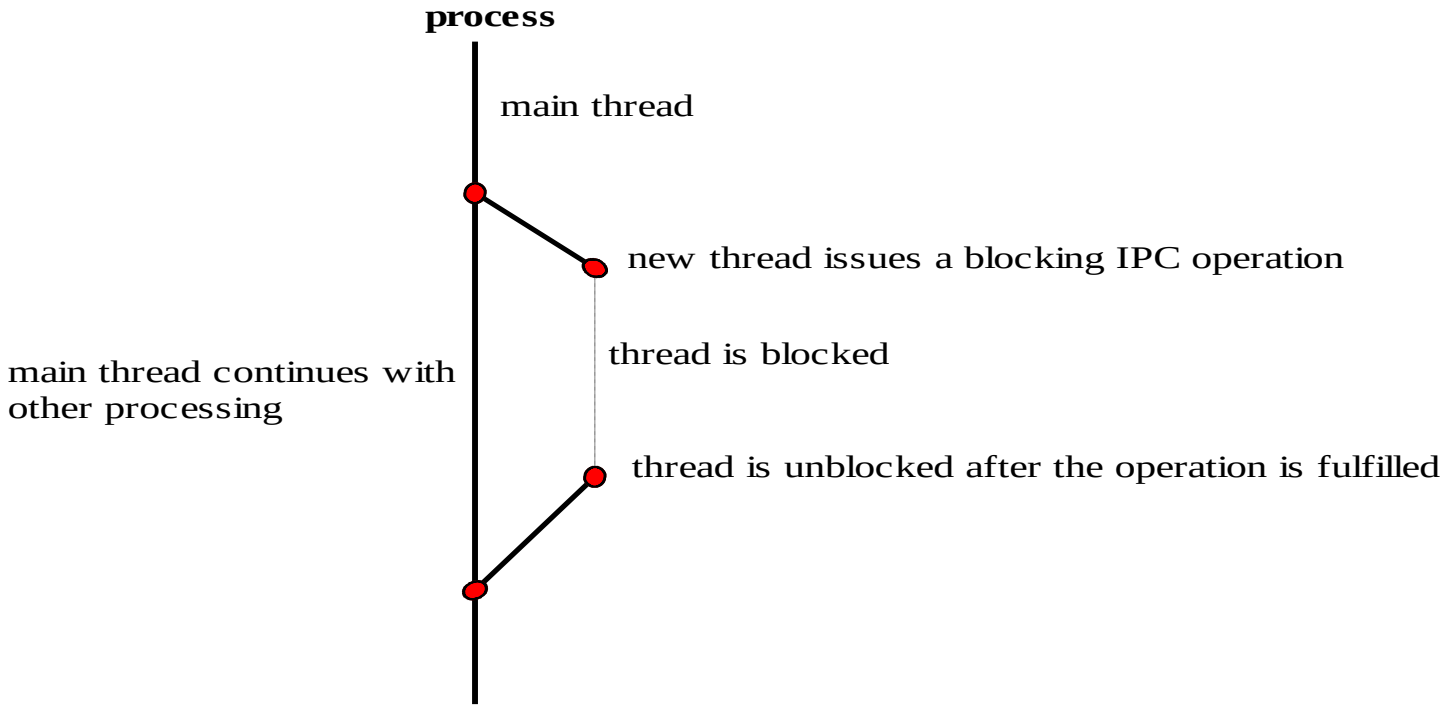
- **Blocking operations issued in the wrong sequence can cause *deadlocks*.**
- **Deadlocks should be avoided. Alternatively, *timeout* can be used to detect deadlocks.**



P1 is waiting for P2's data; P2 is waiting for P1's data.

Using threads for asynchronous IPC

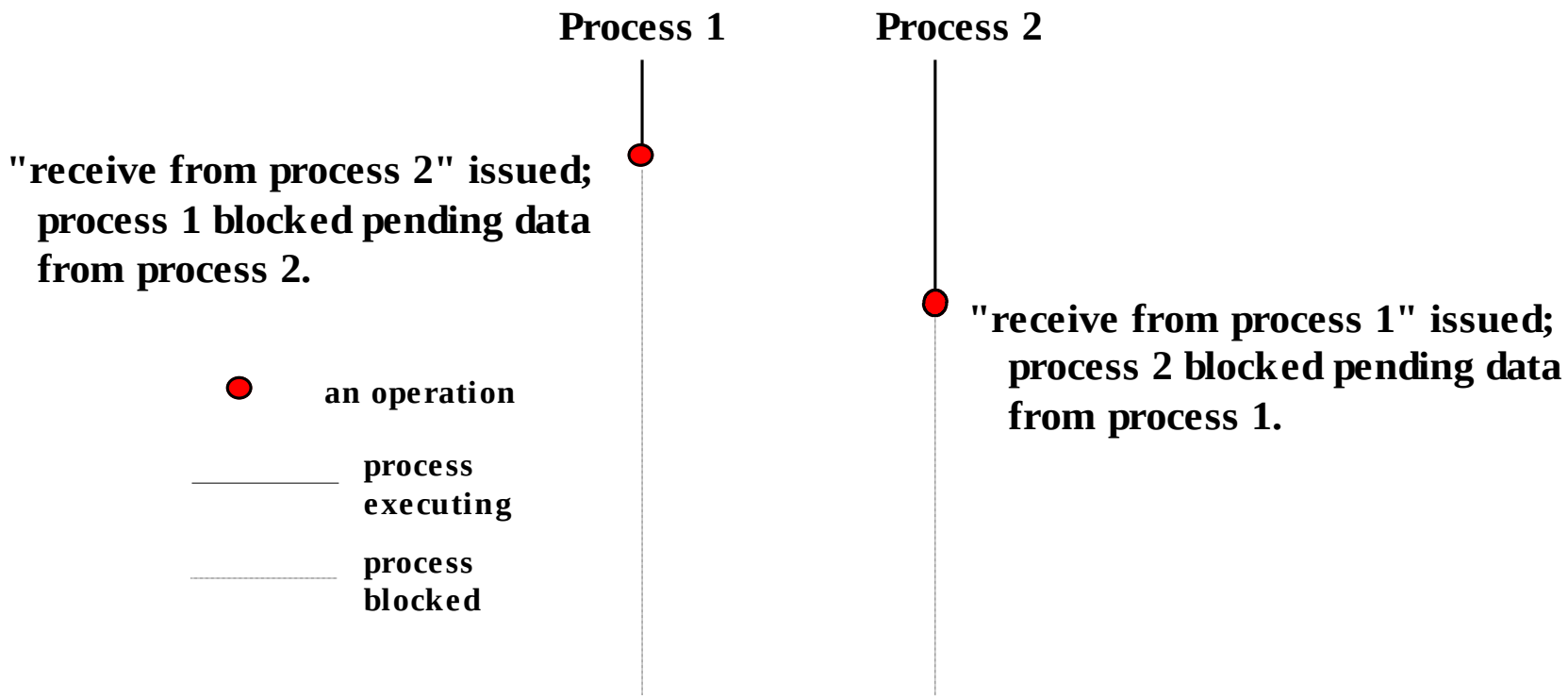
- When using an IPC programming interface, it is important to note whether the operations are synchronous or asynchronous.
- If only blocking operation is provided for send and/or receive, then it is the programmer's responsibility to using child processes or threads if asynchronous operations are desired.



Deadlocks and Timeouts

- Connect and receive operations can result in indefinite blocking
- For example, a blocking *connect request* can result in the requesting process to be suspended indefinitely if the connection is unfulfilled or cannot be fulfilled, perhaps as a result of a breakdown in the network .
- It is generally unacceptable for a requesting process to "hang" indefinitely. Indefinite blocking can be avoided by using **timeout**.
- Indefinite blocking may also be caused by a *deadlock*

Indefinite blocking due to a deadlock



P1 is waiting for P2's data; P2 is waiting for P1's data.

Data Representation

- Data transmitted on the network is a binary stream.
- An interprocess communication system may provide the capability to allow data representation to be imposed on the raw data.
- Because different computers may have different internal storage format for the same data type, an external representation of data may be necessary—standard format.
- *Data marshalling* is the process of (I) flattening a data structure, and (ii) converting the data to an external representation.
- Some well known external data representation schemes are:

  Sun XDR (External Data Representation)

  ASN.1 (Abstract Syntax Notation One)

  XML (Extensible Markup Language)

Data Encoding Protocols

level of abstraction

data encoding schemes	Sample Standards
application specific data encoding language	XML:(Extensible Markup Language)
general data encoding language	ASN.1(Abstract Syntax Notation)
network data encoding standard	Sun XDR(External Data Representation)

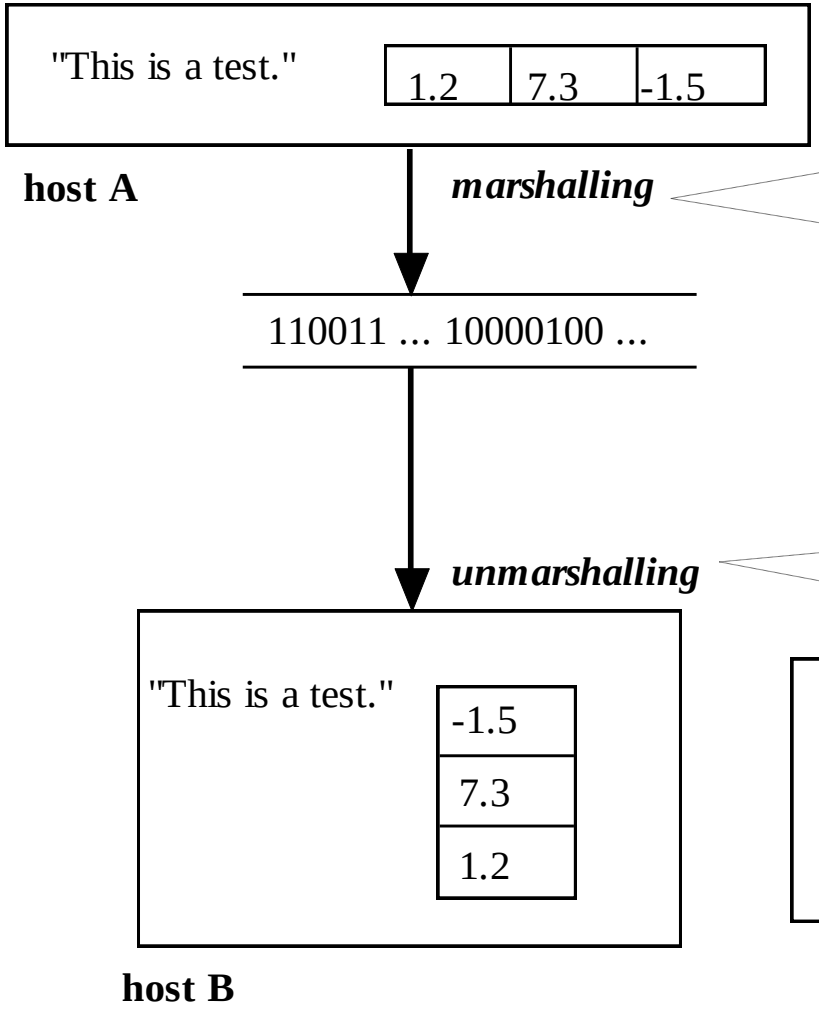
Sample XML file

http://java.sun.com/xml/docs/tutorial/overview/1_xml.html#intro

- XML is a text-based markup language that is fast becoming the standard for data interchange on the Web.
- XML has syntax analogous to HTML.
- Unlike HTML, XML tags tell you what the data *means*, rather than how to display it.
- Example:

```
<message>
        <to>you@yourAddress.com</to> <from>me@myAddress.com</from>
        <subject>XML Is Really Cool</subject>
        <text> How many ways is XML cool? Let me count the ways...
        </text>
</message>
```

Data Marshalling

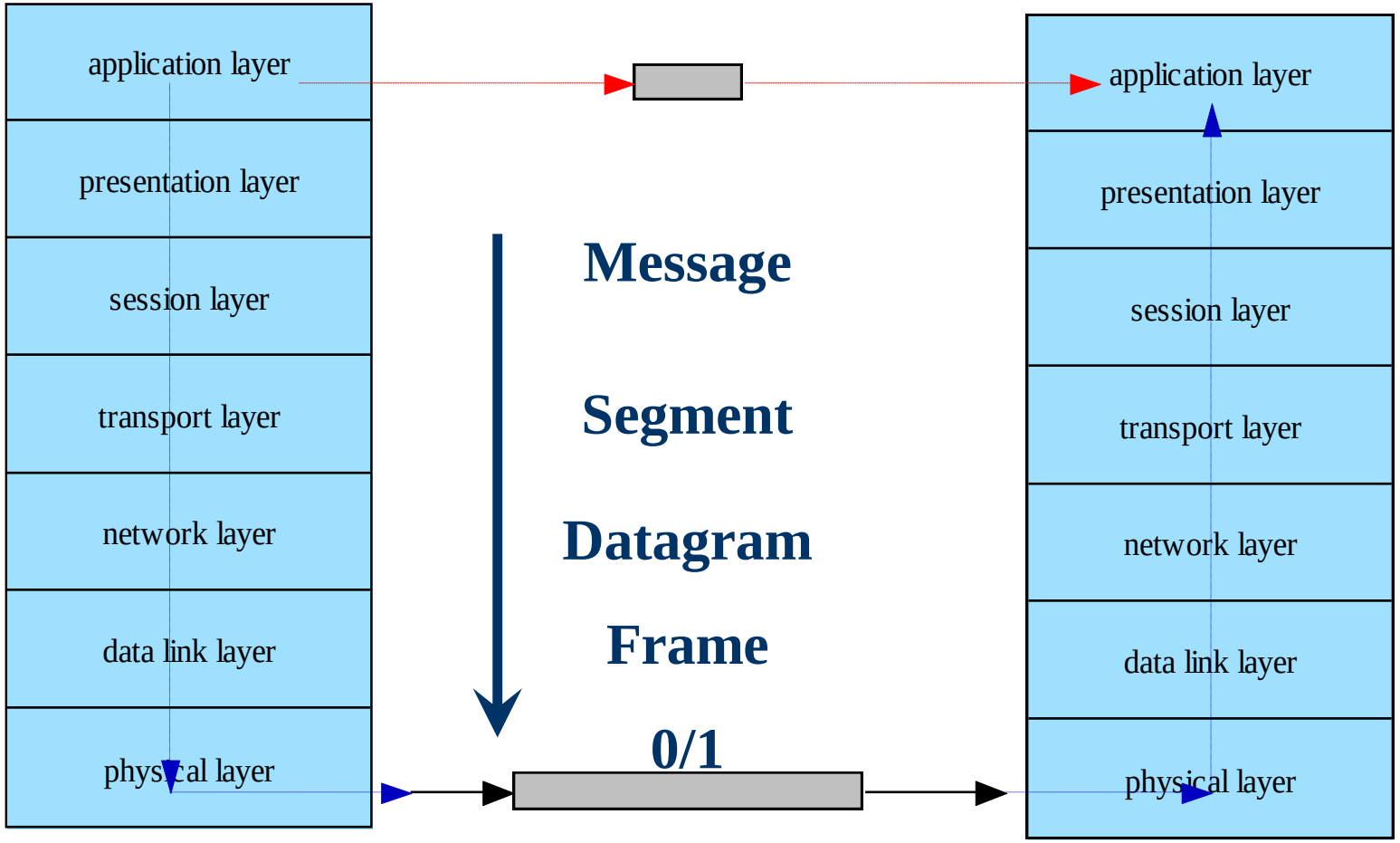


1. flattening of structured data items
2. converting data to external (network) representation

1. convert data to internal representation
2. rebuild data structures.

External to internal representation and vice versa is not required
- if the two sides are of the same host type;
- if the two sides negotiates at connection.

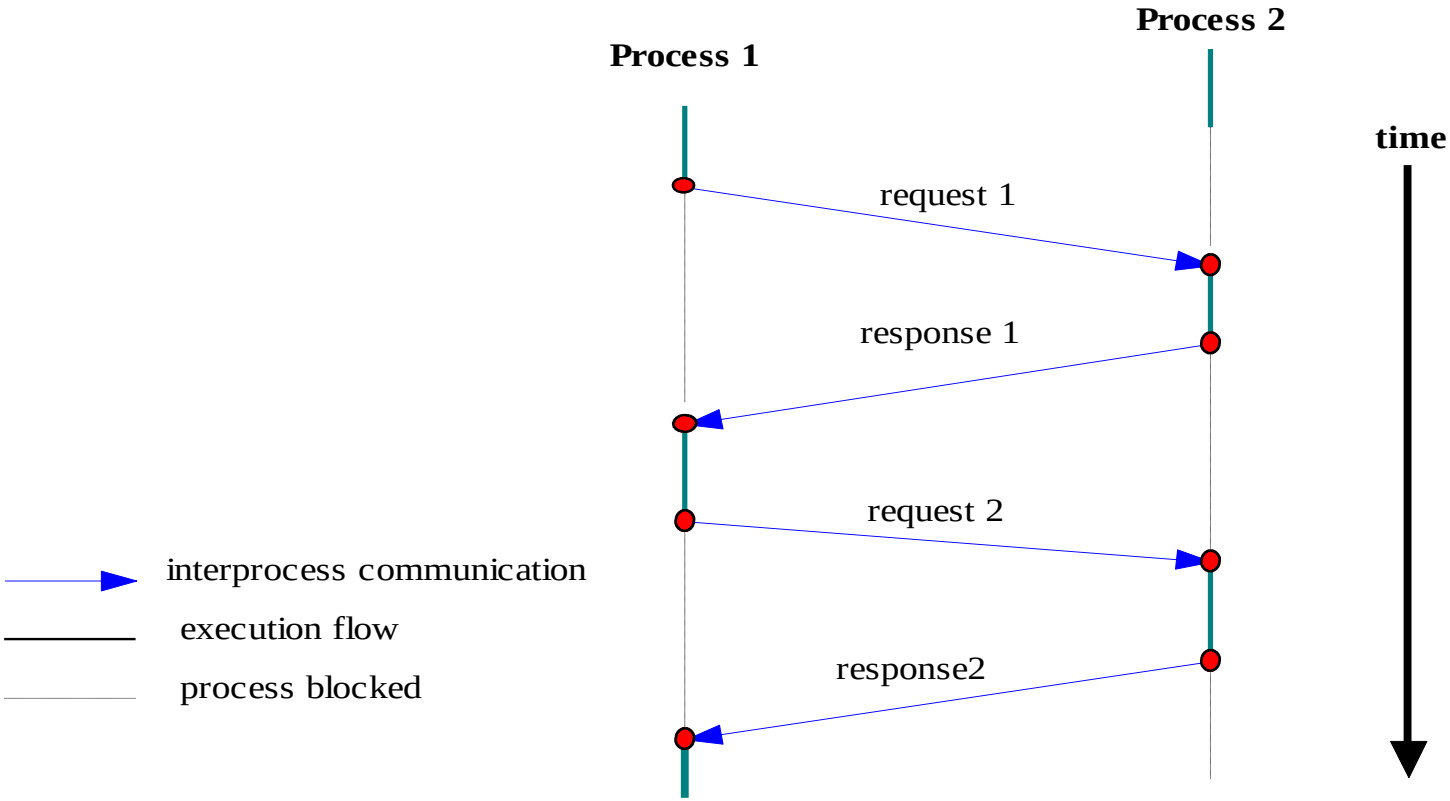
The OSI (Open System Interconnection) Seven-layer network architecture



Text-based protocols

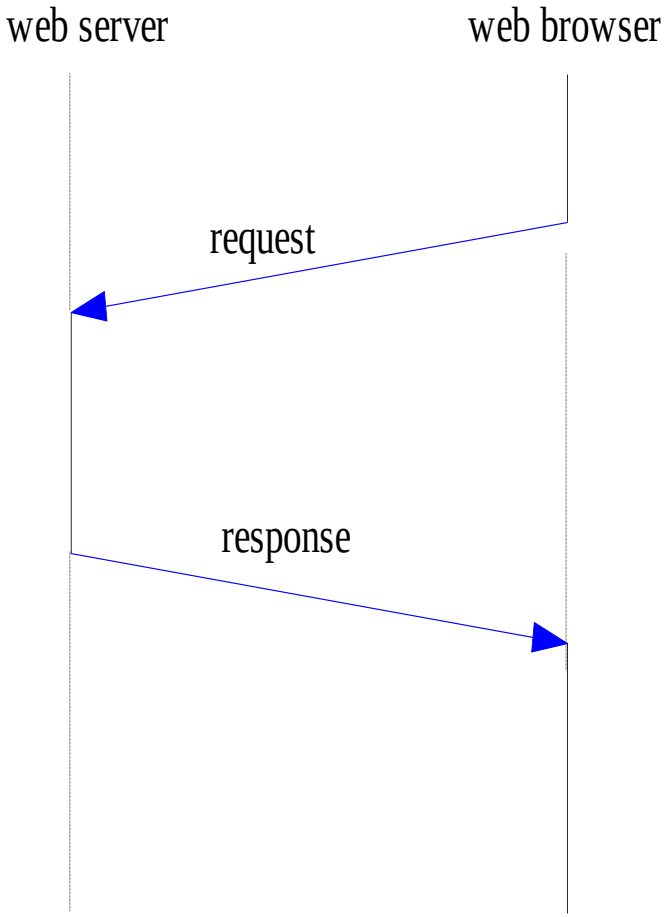
- Data marshalling is at its simplest when the data exchanged is a stream of characters, or text.
- Exchanging data in text has the additional advantage that the data can be easily parsed in a program and displayed for human perusal. Hence it is a popular practice for protocols to exchange requests and responses in the form of character-strings. Such protocols are said to be text-based.
- Many popular network protocols, including FTP (File Transfer Protocol), HTTP, and SMTP (Simple Mail Transfer Protocol), are text-based.

Event diagram



Event diagram for a protocol

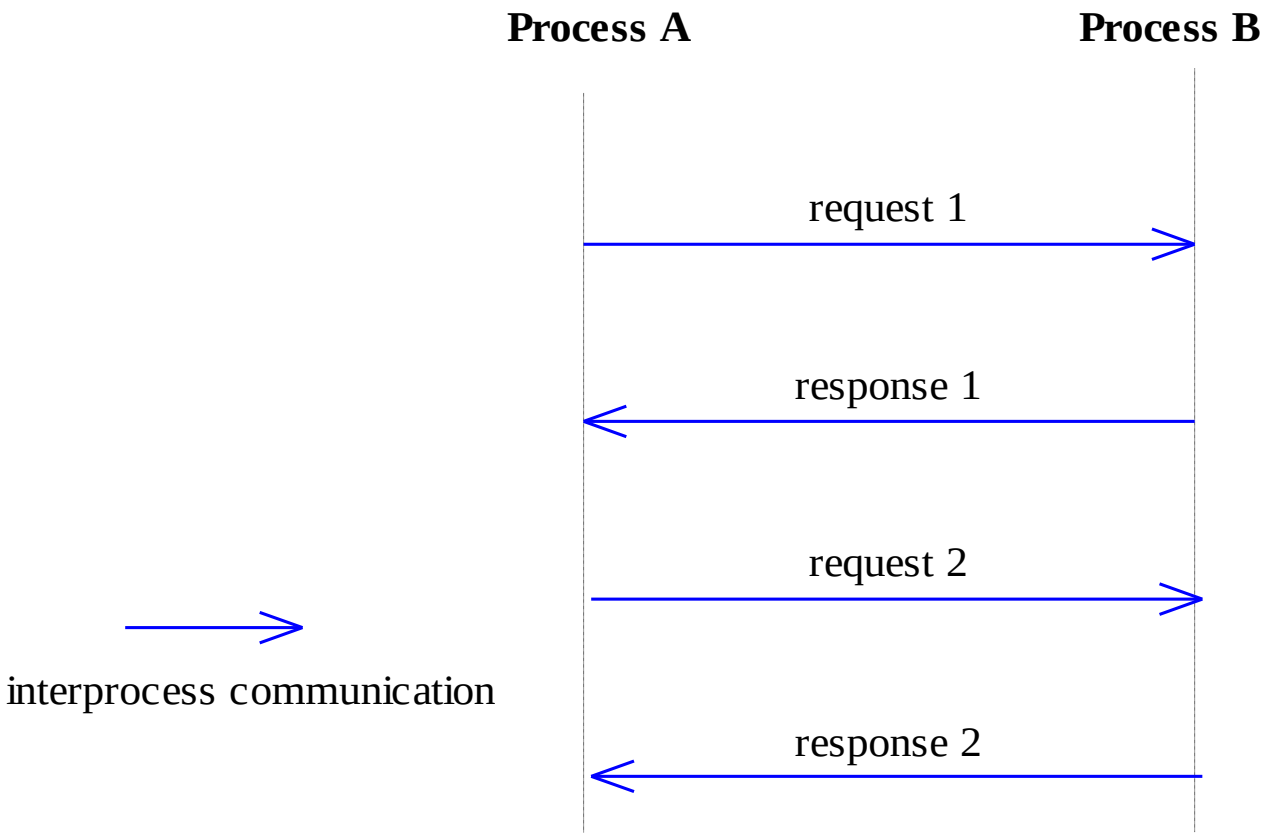
Event Diagram for a HTTP session



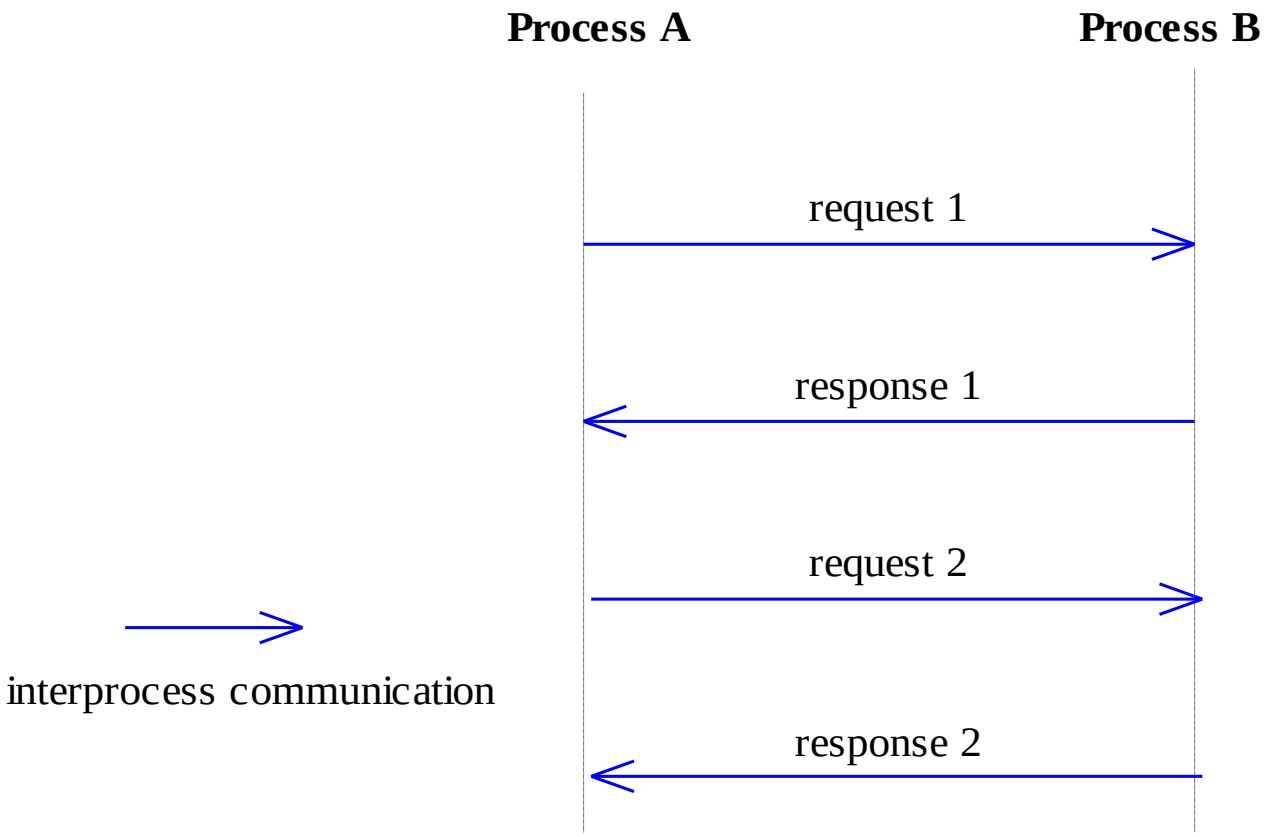
request is a message in 3 parts:
- <command> <document adddress> <HTTP version>
- an optional header
- optional data for CGI data using post method

response is a message consisting of 3 parts:
- a status line of the format <protocol><status code><description>
- header information, which may span several lines;
- the document itself.

Sequence Diagram



Sequence diagram for a HTTP session



Protocol

- In a distributed application, two processes perform interprocess communication in a mutually agreed upon **protocol**.
- The **specification** of a **protocol** should include

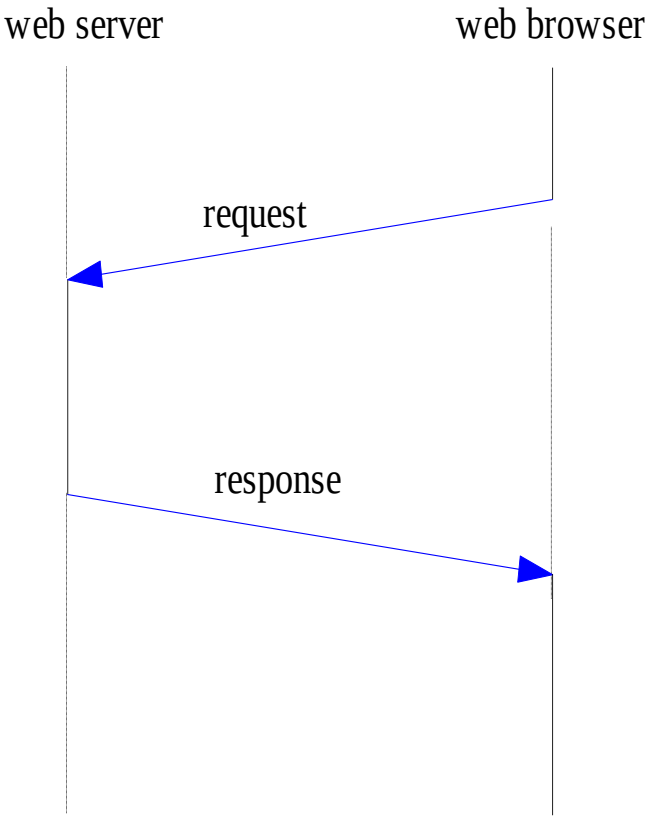
(i) the **sequence** of data exchange, which can be described using a time event diagram.

(ii) the **format** of the data exchange at each step.

HTTP: A sample protocol

- The **H**yper**T**ext **T**ransfer **P**rotocol is a **protocol** for a process (the browser) to obtain a document from a web server process.
- It is a request/response **protocol**: a browser sends a request to a web server process, which replies with a response.

The Basic HTTP protocol



request is a message in 3 parts:
- <command> <document adddress> <HTTP version>
- an optional header
- optional data for CGI data using post method

response is a message consisting of 3 parts:
- a status line of the format <protocol><status code><description>
- header information, which may span several lines;
- the document itself.

We will explore HTTP in details later this quarter.

A sample HTTP session

```
Script started on Tue Oct 10 21:49:28 2000
9:49pm telnet www.csc.calpoly.edu 80
Trying 129.65.241.20...
Connected to tiedye2-srv.csc.calpoly.edu.
Escape character is '^]'.
GET   /~mliu/   HTTP/1.0

HTTP/1.1 200 OK
Date: Wed, 11 Oct 2000 04:51:18 GMT
Server: Apache/1.3.9 (Unix) ApacheJServ/1.0
Last-Modified: Tue, 10 Oct 2000 16:51:54 GMT
ETag: "1dd1e-e27-39e3492a"
Accept-Ranges: bytes
Content-Length: 3623
Connection: close
Content-Type: text/html

<HTML>
<HEAD>
<TITLE> Mei-Ling L. Liu's  Home Page
</TITLE>
</HEAD>
<BODY bgcolor=#ffffff>
 …
```

HTTP Request

HTTP response status line

HTTP response header

document content

HTTP Session

1. Telnet to your favorite Web server:

```
unix> telnet ise.gmu.edu 80
```

Opens a TCP connection to port 80 at ise.gmu.edu.
(default HTTP server port)

2. Type in a GET HTTP request:

```
GET /~yhwang1/ HTTP/1.1
Host: ise.gmu.edu
```

Type above commands and hit carriage return twice, you send this **minimal** but **complete** GET request to HTTP server

3. See what you have in response message sent by HTTP server!

IPC paradigms and implementations

Paradigms of IPC of different levels of abstraction have evolved, with corresponding implementations.

level of abstraction

IPC paradigms	Example IPC Implementations
remote procedure/method	Remote Procedure Call (RPC), Java RMI
socket API	Unix socket API, Winsock
data transmission	serial/parallel communication

UNIX Socket: http://www.ecst.csuchico.edu/~beej/guide/ipc/usock.html